

Image Processing Using OpenCV and CNN Implementation Report

Overview: In this project, I explored convolution in two domains: Image Processing using OpenCV and Convolutional Neural Networks. In Part 1 of the project, the image was processed using OpenCV with filters applied manually to extract features from images, whereas in Part 2, the CNN used filters automatically learned from the CIFAR10 dataset.

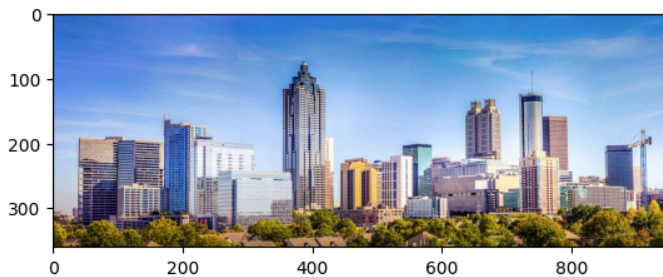
The objective of this report is to understand how convolution operates in both traditional image processing and deep learning, and to compare their effectiveness in feature extraction.

Part 1: Image Processing Using OpenCV

Implementation: Convolution was implemented manually using OpenCV:

- The input RGB image was first converted to grayscale to simplify processing
- Then, blurring filters were applied using averaging kernels of different sizes: 2×2 , 3×3 , 5×5
- The edge detection filters were applied to detect directional features: horizontal edges, vertical edges, and diagonal edges.

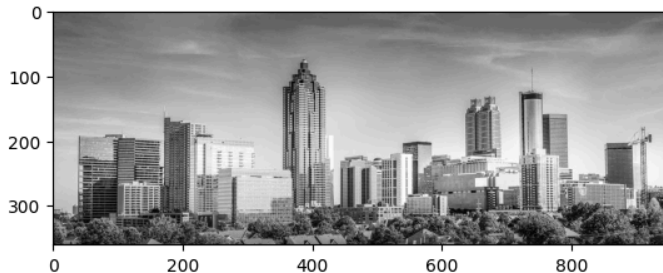
Out[1]: <matplotlib.image.AxesImage at 0x2ba6e2e4490>



Convert the image to grayscale

```
In [2]: # Convert to grayscale for filtering
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
plt.imshow(gray, cmap='gray')
```

Out[2]: <matplotlib.image.AxesImage at 0x2ba6eba1790>

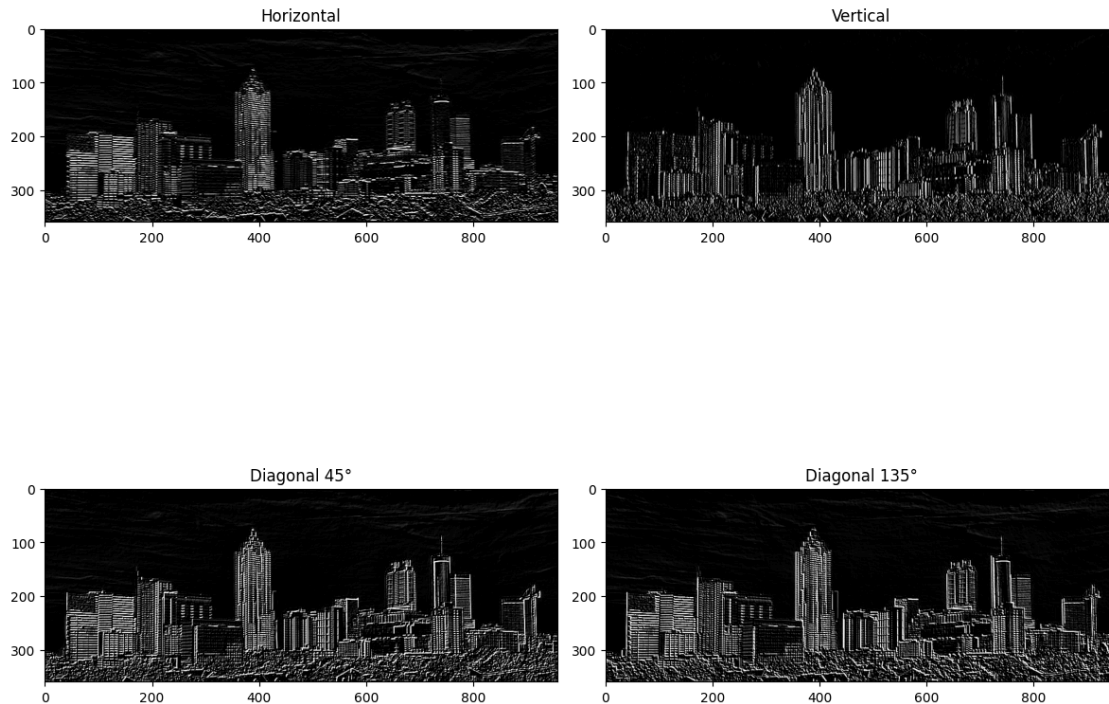


Results: The obtained results are as follows:

- Accuracy: Accurate, as the filters clearly highlighted the prime features

- Time Taken: Around 2 minutes to fully boot up
- Number of Parameters: 2x2: 4 parameters, 3x3: 9 parameters, 5x5: 25 parameters

Findings: I learned that the larger kernels produced stronger blurring but reduced image detail, while the smaller kernels preserved more detail while providing less smoothing. For the edge detection filters, the horizontal filters highlighted flat edges, while the vertical filters emphasized boundaries, and the diagonal filters detected angled patterns.

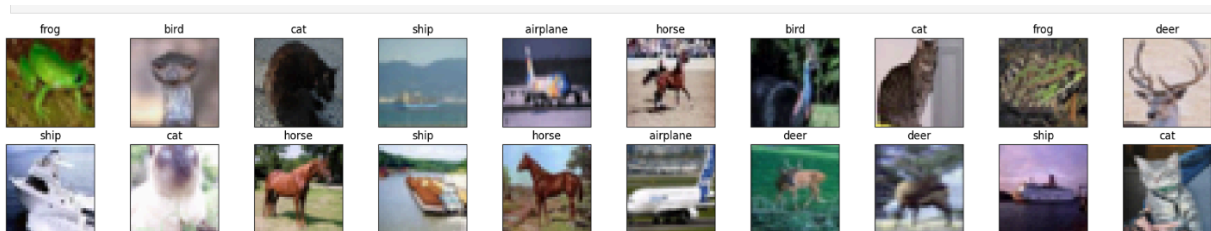


What Helps: By selecting the appropriate kernels for the task and normalizing them, I presume the conversion will be reduced to the computational complexity.

Conclusion: Manual convolution is effective for feature extraction but requires kernel design.

Part 2: Convolutional Neural Networks (CNN)

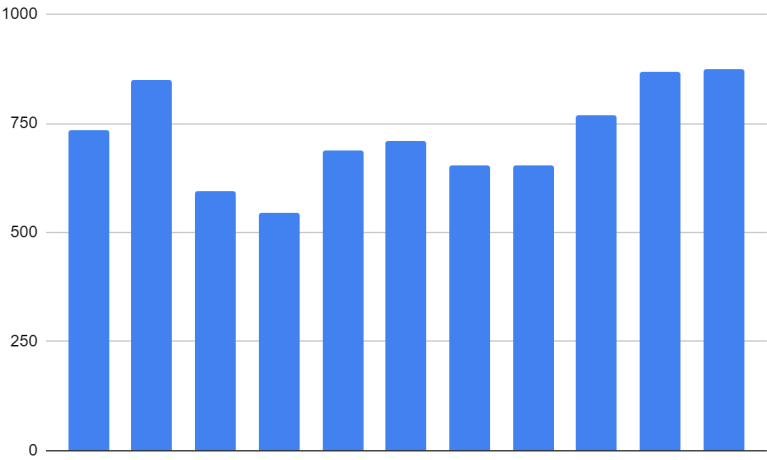
Implementation: A CNN was built and trained on the CIFAR-10 dataset.



Results:

- Accuracy: 70% to 80%
- Time Taken for Training: Approximately 10 to 15 minutes to load up
- Number of Parameters:

Layer	Parameters
Conv1	896
Conv2	18,496
Conv3	73,856
FC1	1,049,088
FC2	5,130



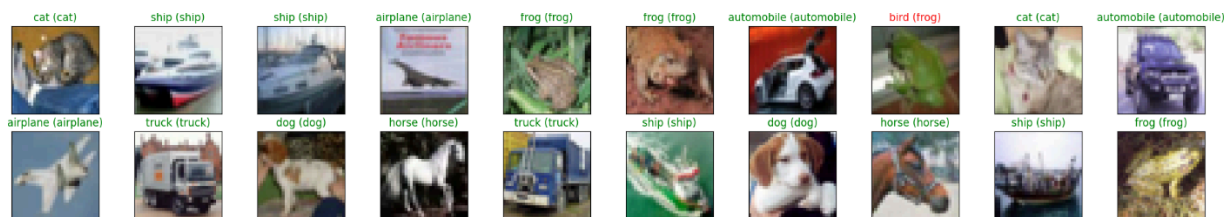
```
Net(  
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  (fc1): Linear(in_features=2048, out_features=512, bias=True)  
  (fc2): Linear(in_features=512, out_features=10, bias=True)  
  (dropout): Dropout(p=0.25, inplace=False)  
)
```

```
Epoch: 1      Training Loss: 1.422041      Validation Loss: 1.127773  
Validation loss decreased (inf --> 1.127773). Saving model ...  
Epoch: 2      Training Loss: 1.024758      Validation Loss: 0.888016  
Validation loss decreased (1.127773 --> 0.888016). Saving model ...  
Epoch: 3      Training Loss: 0.866229      Validation Loss: 0.785632  
Validation loss decreased (0.888016 --> 0.785632). Saving model ...  
Epoch: 4      Training Loss: 0.756061      Validation Loss: 0.761853  
Validation loss decreased (0.785632 --> 0.761853). Saving model ...  
Epoch: 5      Training Loss: 0.684976      Validation Loss: 0.750489  
Validation loss decreased (0.761853 --> 0.750489). Saving model ...
```

Test Loss: 0.775786

Test Accuracy of airplane: 79% (794/1000)
Test Accuracy of automobile: 85% (851/1000)
Test Accuracy of bird: 62% (625/1000)
Test Accuracy of cat: 45% (454/1000)
Test Accuracy of deer: 75% (750/1000)
Test Accuracy of dog: 54% (546/1000)
Test Accuracy of frog: 82% (820/1000)
Test Accuracy of horse: 78% (788/1000)
Test Accuracy of ship: 84% (847/1000)
Test Accuracy of truck: 83% (835/1000)

Test Accuracy (Overall): 73% (7310/10000)



Findings: Validation loss decreased, which indicated the learning was effective. When the model successfully extracted features from images, the performance varied across classes. There was a higher accuracy for structured objects, such as vehicles, and lower accuracy for complex objects, such as animals.

What Helps: Using multiple convolution layers for hierarchical feature extraction and pooling layers to reduce dimensionality and computation.

Conclusion: CNNs learn features at different levels where early layers detect edges, the intermediate layers detect textures, and the deeper layers detect objects. In comparison with OpenCV, it uses manually designed filters, while CNNs learn filters directly from data.

References

- <https://pytorch.org/docs/stable/index.html>
- <https://docs.opencv.org/>
- <https://matplotlib.org/stable/contents.html>